

COMS W4156 Advanced Software Engineering (ASE)

October 31, 2024

On this day ...



Lloyd 1st African-American NBA Player

1950 The Big Cat, [Earl Lloyd](#) becomes the first African-American to play a game in the NBA, scoring 6 points on debut for the Washington Capitols



Basketball Player
[Earl Lloyd](#)

Agenda

1. Team Project Demo: Pineapple
2. CRC Design
3. if time permits,
Second Iteration starts now (reprise)
4. Next



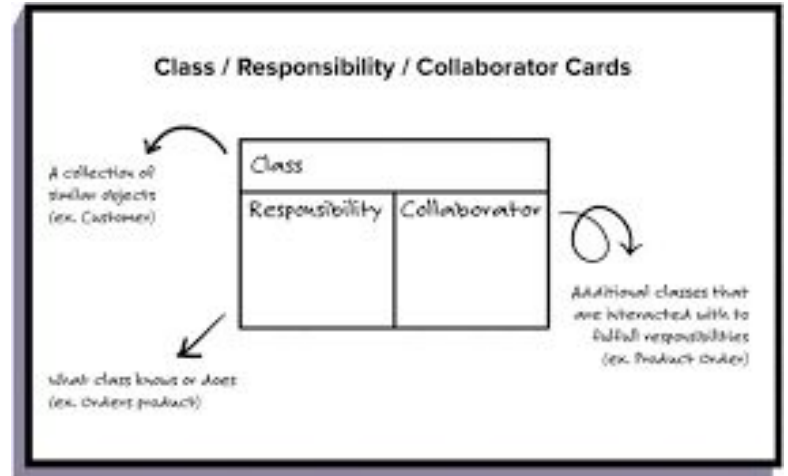
Team Pineapple First Iteration Demo

[demo slides](#)



Agenda

1. Team Project Demo: Pineapple
2. CRC Design
3. if time permits,
Second Iteration starts now (reprise)
4. Next



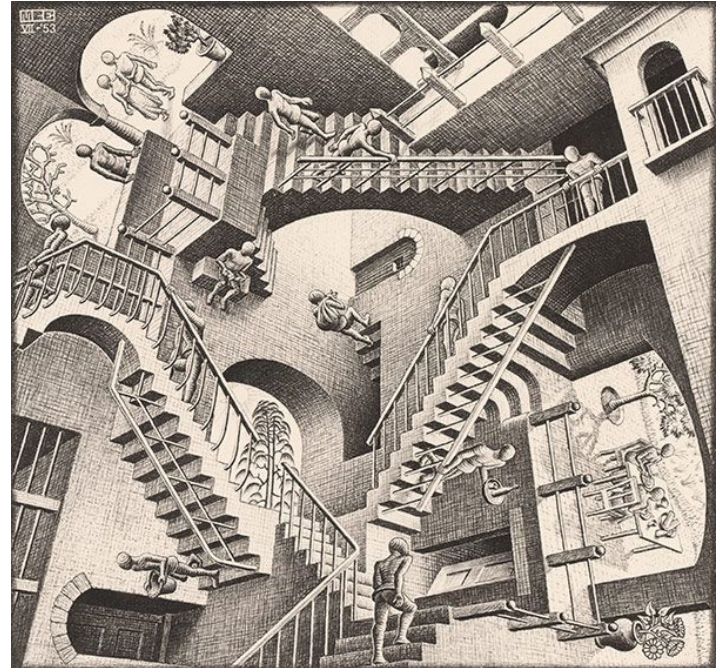
How do you go from User Stories / Use Cases to Code?

- Start coding
- Outsource ([Buy vs Build](#))
- Design ([and Architecture](#))

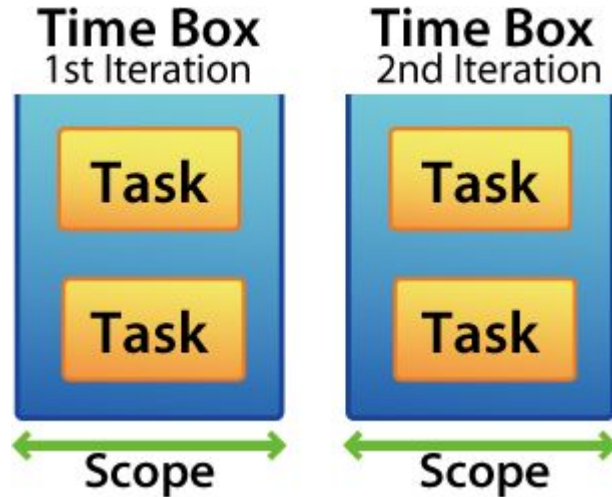
In [agile](#) software development, the current user story "cards" (and their corresponding use cases) should only capture features intended for the current iteration

If there is too much to do, prioritize the user stories and push off the lower priority stories to a later iteration

Example design: [good art](#), [bad code](#)



Agile Time Boxing vs. Scope Boxing



Fixed length iteration (sprint) means scope is limited to what can be accomplished during that time

If more features (scope) are added to an iteration, then the length of the iteration necessarily increases

If less time is allocated to an iteration, then fewer features (scope) can be completed

If it turns out that the originally planned features will take longer than scheduled, then the scope must be reduced with some features pushed off to a later iteration

Agile Design: CRC = Class Responsibilities Collaborators

Like user stories, CRC classes are intended to fit on 3x5 index cards, so often known as “CRC cards”

Each *class* is a thing, entity or object

Each *responsibility* is some task the entity needs to do or data it needs to track

Collaborators are all the other classes the entity interacts with

CRC focuses on the *purpose* of each entity rather than its processes, data flows and data stores (procedural design)

Class Name	
Responsibilities	Collaborators

Find the Classes

Start with the set of use cases

Find the *nouns* other than the actor (role) - classes should be the entities that “do” actions, not the actors who initiate actions

Convert plurals to singular, merge synonyms

Beware adjectives - may mean a whole new class, e.g., “car” vs. “toy car”

Beware hidden nouns, e.g., passive voice

“the thing is activated” = “SOMETHING activates the thing”

Some nouns may be attributes of entities, not themselves entities, e.g., “the radius of a circle” - here the circle is an entity and the radius is an attribute of the circle, it has no meaning as a standalone entity



Find the Classes (Nouns) Example

SEARCH AND REPLACE

A user realizes he mis-capitalized a word everywhere in his document, so he tells the word processor to search for all occurrences of it and replace them with the corrected word.

Class Name	
Responsibilities	Collaborators

Find the Classes (Nouns) Example

Name UC-8: Search and Replace

...

Preconditions A document is loaded and being edited.

Basic Course of Events

1. The user indicates that the software is to perform a search-and-replace in the document.
2. The software responds by requesting the search term and the replacement text.
3. The user inputs the search term and replacement text and indicates that all occurrences are to be replaced.
4. The software replaces all occurrences of the search term with the replacement text.

...

Postconditions All occurrences of the search term have been replaced with the replacement text.

Class Name	
Responsibilities	Collaborators

Find the Responsibilities

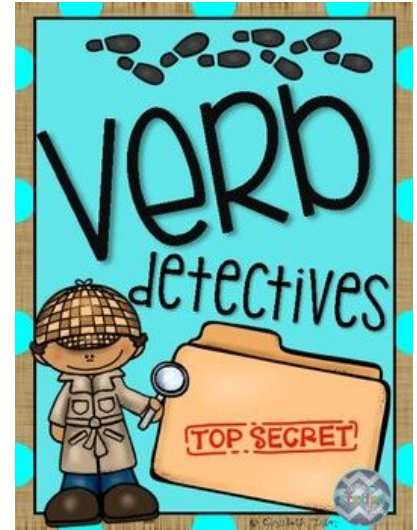
Find the *verbs* (some may be CRUD)

Say WHAT gets done, not HOW it gets done

Turn passive verbs into active verbs

“the thing is activated” = “SOMETHING activates the thing”

The classes and responsibilities create a vocabulary for discussing the design



Find the Responsibilities (Verbs) Example

SEARCH AND REPLACE

A user realizes he mis-capitalized a word everywhere in his document, so he tells the word processor to search for all occurrences of it and replace them with the corrected word.

Class Name	
Responsibilities	Collaborators

Find the Responsibilities (Verbs) Example

Name UC-8: Search and Replace

...

Preconditions A document is loaded and being edited.

Basic Course of Events

1. The user indicates that the software is to perform a search-and-replace in the document.
2. The software responds by requesting the search term and the replacement text.
3. The user inputs the search term and replacement text and indicates that all occurrences are to be replaced.
4. The software replaces all occurrences of the search term with the replacement text.

...

Postconditions All occurrences of the search term have been replaced with the replacement text.

Class Name	
Responsibilities	Collaborators

Find the Collaborators



Find any relationships or associations among entities - not necessarily symmetric, can be one-way

A Collaborator is any entity that a class needs to know about, communicate with, contain, control, or that otherwise helps it perform one or more responsibilities that it cannot perform on its own

If that entity does not already have its own CRC card, make a new card

If that entity does not already have a responsibility to do something or provide something on behalf of the class at hand, add that responsibility (and any additional collaborators it needs)

Find the Collaborators (Relationships among Nouns) Example

SEARCH AND REPLACE

A user realizes he mis-capitalized a word everywhere in his document, so he tells the word processor to search for all occurrences of it and replace them with the corrected word.

Class Name	
Responsibilities	Collaborators

Find the Collaborators (Relationships among Nouns) Example

Name UC-8: Search and Replace

...

Preconditions A document is loaded and being edited.

Basic Course of Events

1. The user indicates that the software is to perform a search-and-replace in the document.
2. The software responds by requesting the search term and the replacement text.
3. The user inputs the search term and replacement text and indicates that all occurrences are to be replaced.
4. The software replaces all occurrences of the search term with the replacement text.

...

Postconditions All occurrences of the search term have been replaced with the replacement text.

Class Name	
Responsibilities	Collaborators

CRC Design Process

Develop a set of CRC cards for the current set of user stories / use cases

CRC does not need an object-oriented programming language - CRC can stand for *Component* Responsibilities Collaborators

Walk through the steps and flows of each use case and *animate* the CRC cards: move index cards on a table or sticky notes on a board



Make sure every behavior is a responsibility of some class, and figure out what each class needs from collaborators to fulfill its responsibilities

There is no Global Master



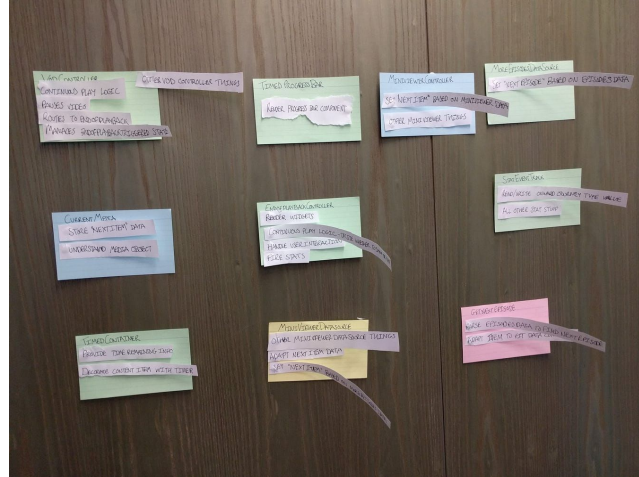
When “the system” does something, that means some class in the system does it

There should not be any global master that does everything and/or is related to everything else

Give up global knowledge of control and rely on local knowledge of objects to accomplish their tasks

Debug the CRC cards

- Remember classes are nouns that “do” operations, not the actors who initiate operations
- Split classes with too many responsibilities
- Combine classes with too few responsibilities
- Remove redundancy



NOW Start Coding

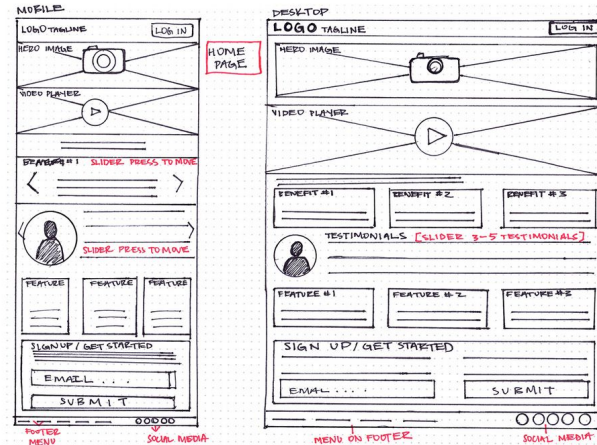
Class -> class, module, file

Responsibility -> method, function, subroutine

Collaborator -> calls, parameters, shared data

GUI applications:

wireframes might or might not
be developed before coding



Example CRC Cards

[ATM](#)

[Doctors Office](#)

[Drawing program](#)



Agenda

1. Team Project Demo: Pineapple
2. CRC Design
3. if time permits,
Second Iteration starts now (reprise)
4. Next

The text "SECOND ROUND" is rendered in a pixelated, blocky font. "SECOND" is in blue with a black outline, and "ROUND" is in orange with a black outline. The letters have a 3D effect with a dark shadow underneath.

Second Iteration

Due November 25 - 4 weeks to build version 2 and sample client

Version 2 of your service should be your final version (except for bug fixes)



Remember the Team Project Constraints



Your team's service must be a service, not a library and not an app

The first iteration focused entirely on the service, the second iteration adds a client that uses the service's API. The client is code, not human! Your service's human user is the developer who writes this client code

The client may be either a service or an app (or even a library, if you can find some way to demo).

Your service must be developed in Java or C++, Python is not allowed except for AI components (if any)

The client depends on the service but the service must not depend on the client

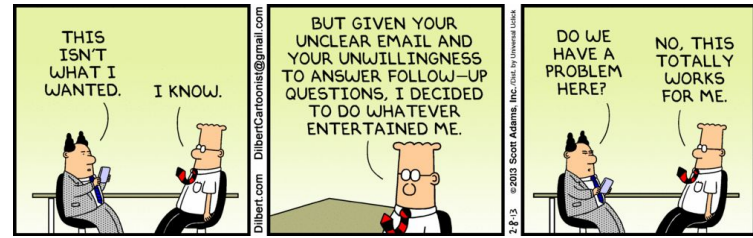
The client must target an acceptable user group that is not university students, staff or faculty (except for individuals who happen to also be members of an acceptable user group)

What Is An Acceptable User Group?



- Elderly (defined for this purpose as age 75 or over)
- Disabled or impaired (e.g., vision, hearing, speech, motor, cognitive)
- Healthcare workers (doctors, medical technicians, nurses, aides, etc.)
- First responders (police, fire, EMT)
- Indigenous peoples ("distinct social and cultural groups that share collective ancestral ties to the lands and natural resources where they live, occupy or from which they have been displaced")
- Homeless people ("lacks a fixed, regular, and adequate nighttime residence")
- Maybe others with **societal benefits** - ask me *in advance*

Second Iteration Overview



Final version of your service's API. If the API is not the same as proposed in your revised proposal, your README should explain what is different and why

Demoable client that uses your service's API and targets one (or more) of the designated target audiences

Service supports multiple instances of the same client running simultaneously (show this in your demo)

Client code should reside in the same repository as your service or in another repository linked from the README of your service repository. Make sure the instructor and your Mentor are collaborators on both repos

Second Iteration Testing



Update unit test suite (with mocking) to run under CI

Update API test suite to run under CI and exercise your service's complete final API, including support for multiple clients and persistent data

Add internal integration testing to run under CI to cover direct and indirect dependencies within your service implementation

Add external integration testing to run under CI to cover dependencies on third-party code (if any) and external resources (files, databases, devices if applicable) within your service implementation

Collectively, your automated tests under CI should achieve $\geq 85\%$ branch coverage

Add end-to-end testing where your client exercises your service's API, manual is ok

Second Iteration Documentation



Document what your client does and how it helps its target audience

Document your API entry points, including ordering rules and all possible errors and exceptions - either prose or a special format like OpenAPI/Swagger is ok

Document both your own client's use of your service's API and what a third party would need to do (and not do) to develop their own client that uses your service's API

Document your tests, particularly integration tests - which specific interfaces are they testing? Where did the inputs come from? How do you know the outputs and side-effects are correct? Consider all inputs and outputs, such as shared data, not just parameters and return values

Second Iteration Odds and Ends

Make sure everything that matters is in the main branch of your repository(ies), your Mentor will not check other branches

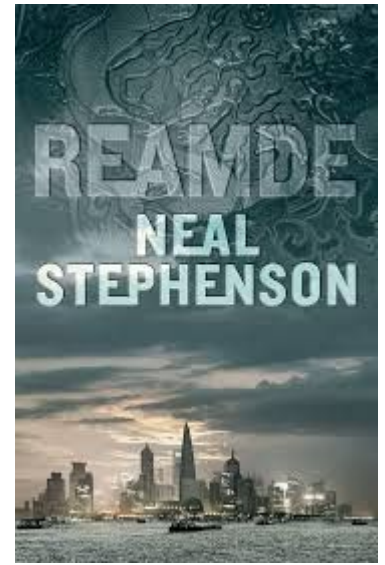
Your README(s) should include instructions about how to build and run both your service and your client, and instructions about how to perform any manual testing

Your main README should include a link to your project management task board

Run style checker and static analysis bug finder under CI on your entire service codebase (not required for client) and fix as many flaws as possible

Include test runner, coverage, static checker, etc. reports in your service repository

Explain any third-party code in your repository(ies): what is it, where did you get it, what are you using it for?



Second Iteration Demo

Due December 6

Demo for your mentor sometime during the approx. two weeks following the second iteration deadline (note Thanksgiving break is in the middle)

You can continue working on your project between the second iteration deadline and the second iteration demo (and Demo Day later), but focus on fixing bugs and presenting an impressive demo



Agenda

1. Team Project Demo: Pineapple
2. CRC Design
3. if time permits,
Second Iteration starts now (reprise)
4. **Next**



Rest of Semester Milestones



[Second Iteration](#) due November 25 (Monday before Thanksgiving)

[Second Iteration Demo](#) due December 6 (Friday the week after Thanksgiving)

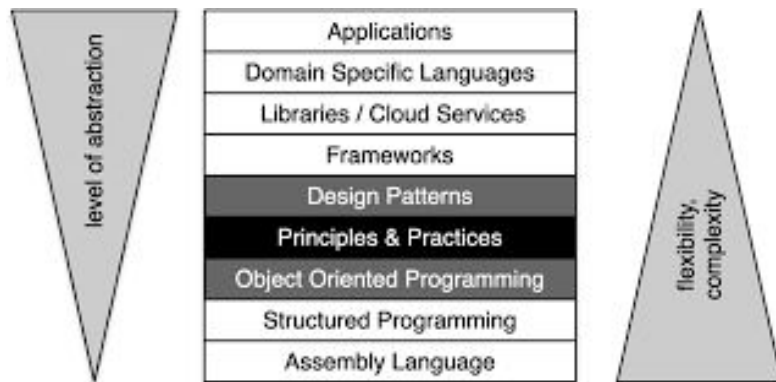
Second Assessment Friday December 6 12:01AM to Monday December 9 11:59PM (last day of classes and the day before university "study days")

[Demo Day](#) Monday December 16 10AM to 4PM (Monday of finals week)

Next Week

!No Class on Tuesday! (university holiday)

Thursday we start Software Design Patterns and Principles





Ask Me Anything

